

Reviewer Pack — Roe-Skeleton Spectral Gap Forces Multiplicity in Protocol-In...

Entry a510fd856726 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 14:45:39 UTC

Roe-Skeleton Spectral Gap Forces Multiplicity in Protocol-Induced Covers on Small-Diameter Gadget Lifts

Entry ID: a510fd856726 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 14:45:39 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Lifting (CGL) — Coarse Geometry / Roe-Algebra Operator Theory applied to Lifting in Communication Complexity **Field B** (complexity object): Deterministic two-party communication complexity of $f \circ G^n$, specifically the multiplicity profile m_Π of the ProtocolPullback partition on the LiftedInputSpace $(X \times Y)^n$

Statement:

Let $G = (X, Y, g, d)$ be a MetricGadget with $R := \text{diam}(G)$ and let A_R denote the 0/1 adjacency matrix of the RoeAlgebraSkeleton at scale R on $(X \times Y)^n$ under the $d_\oplus$ metric, with combinatorial Laplacian $L_R = D_R - A_R$. Let $\lambda_R(n)$ be the smallest nonzero eigenvalue of L_R , and let $N = |X \times Y|^n$. Then for every deterministic communication protocol Π computing any non-constant $f \circ G^n$, the ProtocolPullback multiplicity satisfies $m_\Pi \cdot R_\Pi \geq \lceil \lambda_R(n) \cdot N / (4 \cdot \max\text{-degree}(A_R)) \rceil$, and consequently the CoarseLiftingComplexity invariant obeys $\text{CLC}(f, G) \geq \log_2(\lambda_R(n) \cdot N / (4 \cdot \deg))$ for every f with $D(f) \geq 1$. Equivalently, a vanishing Roe-skeleton spectral gap is a coarse obstruction that lower-bounds protocol cover multiplicity, in alignment with axiom A2 (Protocol $\rightarrow$ Cover) and A4 (Property-A Transfer).

Rationale (proposer's reasoning):

This concretizes axiom A2 by predicting a quantitative spectral certificate: the Roe algebra skeleton's Cheeger-type gap controls the minimum multiplicity any transcript-induced cover must exhibit at scale R . The Cheeger inequality on the skeleton graph implies the boundary-to-bulk ratio of any low-diameter set is at least $\lambda_R/(2 \cdot \deg)$; since a transcript pullback of scale R_Π and multiplicity m_Π yields a cover where each cell has $d_\oplus$ -diameter $\leq R_\Pi$, the spectral gap forces the cover cells to be 'fat' — i.e., to overlap. A protocol with multiplicity 1 (a partition) at scale R is precluded whenever $\lambda_R > 0$, certifying that $\text{asdim} \geq 1$ in the lifted geometry. This also gives a lattice on which Property-A (axiom A4) can be cross-checked, since amenable/Property-A spaces have shrinking gaps but persistent multiplicity, providing a concrete bridge between coarse spectral data and communication lower bounds.

Taxonomy category: LIFTING (status at proposal time:)

Framework membership: framework fw_6997a27304, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 868dbbafafa3af6f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across both gadgets G_{XOR} and G_{IND} , for $n \in \{1, 2, 3\}$ and every non-constant Boolean f on n bits with both canonical and alternative protocols Π , the inequality $m_{\Pi} \cdot R_{\Pi} \geq [\lambda_R(n) \cdot N / (4 \cdot \text{max-degree})]$ must hold with zero violations; at least one (n, gadget) pair must yield a nontrivial $RHS \geq 2$; $\lambda_R(n) > 0$ in all cases. Single deterministic run (no seeds; computation is exact).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Roe algebra coarse geometry communication complexity lifting - spectral gap Laplacian lower bound deterministic communication protocol partition-property A coarse embedding gadget composition lifting theorem

Top relevant hits considered: - [<http://arxiv.org/abs/1903.06084v8>] Lifting coarse homotopies - [<http://arxiv.org/abs/2312.08907v2>] Roe algebras of coarse spaces via coarse geometric modules - [<http://arxiv.org/abs/2408.07701v2>] A categorical interpretation of Morita equivalence for dynamical von Neumann algebras - [<http://arxiv.org/abs/2403.19550v2>] Spectral gap for surfaces of infinite volume with negative curvature - [<http://arxiv.org/abs/1806.02734v3>] Spectral lower bounds for the orthogonal and projective ranks of a graph - [<http://arxiv.org/abs/2404.02414v2>] A simple lower bound for the complexity of estimating partition functions on a quantum computer - [<http://arxiv.org/abs/1411.4413v2>] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data - [<http://arxiv.org/abs/0901.0512v4>] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict
```

Constants and utility functions

```

def hamming_distance(u, v):
    return sum(1 for x, y in zip(u, v) if x != y)

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    rank = 0
    pivot_col = 0
    pivot_row = 0

    while pivot_col < n and pivot_row < m:
        # Find the pivot row
        max_idx = pivot_row
        for i in range(pivot_row + 1, m):
            if abs(A[i][pivot_col]) > abs(A[max_idx][pivot_col]):
                max_idx = i

        if A[max_idx][pivot_col] == 0:
            pivot_col += 1
            continue

        # Swap rows to put the pivot in place
        A[pivot_row], A[max_idx] = A[max_idx], A[pivot_row]

        # Eliminate entries below the pivot
        for i in range(pivot_row + 1, m):
            factor = -A[i][pivot_col] / A[pivot_row][pivot_col]
            for j in range(n):
                A[i][j] += factor * A[pivot_row][j]

        rank += 1
        pivot_row += 1
        pivot_col += 1

    return rank

def matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0 for _ in range(n)] for _ in range(m)]

    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]

    return C

def max_degree(A):
    return max(sum(row) for row in A)

# MetricGadget definitions
X_XOR = ['0', '1']
Y_XOR = ['0', '1']
d_XOR = hamming_distance

```

```

R_XOR = 2

X_IND = ['0', '1']
Y_IND = ['0', '1', '10', '11']
d_IND = hamming_distance
R_IND = 2

# ProtocolPullback computation

def protocol_pullback(protocol, gadget, n):
    X, Y, d, R = gadget
    N = len(X) ** n * len(Y) ** n
    max_degree_A = max_degree(A_R)

    m_Π = 0
    R_Π = 0

    for f in range(2**n):
        transcript_cells = set()
        for x in X:
            for y in Y:
                cell = (x, y)
                if protocol(f, cell):
                    transcript_cells.add(cell)

        m_Π = max(m_Π, len(transcript_cells))
        R_Π = max(R_Π, max(hamming_distance(u, v) for u, v in itertools.combinations(transcript_cells, 2)))

    return m_Π, R_Π

def canonical_protocol(f, cell):
    x, y = cell
    return f(x) == y

def alternative_protocol(f, cell):
    x, y = cell
    return f(x) != y

# run_trial function

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for gadget, X, Y, d, R in [(X_XOR, Y_XOR, d_XOR, R_XOR), (X_IND, Y_IND, d_IND, R_IND)]:
        for n in [1, 2, 3]:
            A_R = [[0] * len(X) ** n * len(Y) ** n for _ in range(len(X) ** n * len(Y) ** n)]

            for u in range(len(X) ** n):
                for v in range(len(Y) ** n):
                    if hamming_distance((u // len(Y) ** n, u % len(Y) ** n), (v // len(Y) ** n, v % len(Y) ** n)) == d:
                        A_R[u][v] = 1

            L_R = [[0] * len(A_R) for _ in range(len(A_R))]
            degree = [sum(row) for row in A_R]

```

```

    for i in range(len(L_R)):
        L_R[i][i] = -degree[i]
        for j in range(i + 1, len(L_R)):
            if A_R[i][j]:
                L_R[i][j] = A_R[j][i] = 1

    eigenvalues = [0] * 2
    eigenvectors = [[0] * len(A_R) for _ in range(2)]

    # Compute the smallest nonzero eigenvalue and eigenvector using power iteration
    v = [random.random() for _ in range(len(A_R))]
    for _ in range(100):
        v = matrix_multiply(A_R, v)
        v = [x / math.sqrt(sum(x**2 for x in v)) for x in v]

    eigenvalues[0] = max(v[i] * A_R[i][j] * v[j] for i in range(len(A_R)) for j in range(len(A_R)))
    eigenvectors[0] = v

    # Compute the multiplicity and scale
    m_Π, R_Π = protocol_pullback(canonical_protocol, (X, Y, d, R), n)
    m_Π_alt, R_Π_alt = protocol_pullback(alternative_protocol, (X, Y, d, R), n)

    # Check the inequality
    if m_Π * R_Π < math.ceil(eigenvalues[0] * len(X) ** n * len(Y) ** n / (4 * max_degree)):
        results.append(("m_Π={m_Π}, R_Π={R_Π}, λ_R(n)={eigenvalues[0]}, N={len(X) ** n * len(Y) ** n}"))
    else:
        results.append(("m_Π={m_Π}, R_Π={R_Π}, λ_R(n)={eigenvalues[0]}, N={len(X) ** n * len(Y) ** n}"))

    metric_value = sum(1 for _, valid in results if not valid)
    conjecture_holds = all(not valid for _, valid in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "ProtocolPullbackMultiplicity",
        "metric_value": metric_value,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

# Main execution

if __name__ == "__main__":
    seeds = [11, 23, 37, 53, 71] if not sys.argv[1:] else [int(s) for s in sys.argv[1:]]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if not r["conjecture_holds"]) / len(results)

    if support_fraction == 0:

```

```

    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = seeds[next(i for i, r in enumerate(results) if not r["conjecture_holds"])]
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7ce6686.py", line 178, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7ce6686.py", line 119, in run_trial
    for gadget, X, Y, d, R in [(X_XOR, Y_XOR, d_XOR, R_XOR), (X_IND, Y_IND, d_IND, R_IND)]:
                              ~~~~~
ValueError: not enough values to unpack (expected 5, got 4)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ValueError (tuple unpacking mismatch) before producing any data, so neither the support nor the falsification condition can be evaluated. | next: Fix the iterable in run_trial to include the gadget label (5-tuple) and rerun the deterministic test to obtain $\lambda_R(n)$, N, and max-degree values.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	7600
2	novelty	claude_max	opus	0	0	3811
3	novelty	claude_max	opus	0	0	10318
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	60068
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20515
6	judge	claude_max	opus	0	0	4371

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 106682 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/a510fd856726.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a510fd856726.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a510fd856726.tar.gz` (if generated)